

- **Laravel Relationships**
- **Faker Library**

1-Laravel Relationships:

ال Relationships في Laravel تستخدم عشان تربط الجداول ببعض باستخدام Eloquent ORM.

1-One To One

هي علاقة يكون فيها سجل واحد من جدول مرتبط بسجل واحد فقط من جدول آخر.

مثال: كل User له Profile واحد.

User- يمتلك Profile واحد.

Profile ينتمي إلى User واحد.

Laravel Methods: hasOne() - belongsTo()

// user model :

```
public function profile(){ return $this->hasOne(Profile::class);}
```

//prof model

```
public function user(){ return $this->belongsTo(User::class);}
```

تساعد على فصل البيانات الإضافية في جدول مستقل مع الحفاظ على الارتباط بينها.

2-One To Many Relationship

هي علاقة يكون فيها سجل واحد من جدول مرتبط بعدة سجلات من جدول آخر. ينما كل سجل من الجدول الثاني ينتمي إلى سجل واحد فقط من الجدول الأول.

مثال:

- User واحد يمكن أن يكتب العديد من Posts.
- كل Post ينتمي إلى User واحد فقط.
- Laravel Methods: hasMany() - belongsTo()
- تسمح بتنظيم البيانات وتقليل التكرار داخل قاعدة البيانات.

3-Many To Many Relationship

هي علاقة يمكن فيها لكل سجل من الجدول الأول أن يرتبط بعدة سجلات من الجدول الثاني والعكس صحيح. -عندما يكون الارتباط متبادلاً ومتعددًا بين الجدولين.

مثال:

- Post يمكن أن ينتمي إلى عدة Categories.
- Category يمكن أن تحتوي على عدة Posts.

تحتاج إلى جدول وسيط (Pivot Table) مثل: category_post

Laravel Methods: belongsToMany()

تسهل إدارة العلاقات المتعددة بين الجداول دون تكرار البيانات

4-Eager Loading :

تقنية تُستخدم لتحميل العلاقات مع البيانات الأساسية في استعلامات أقل لتجنب مشكلة N+1 Problem وتحسين الأداء.

هي مشكلة تحدث عندما يقوم Laravel بتنفيذ استعلام رئيسي ثم ينفذ استعلامًا إضافيًا لكل سجل للحصول على العلاقات المرتبطة به.

Post::with('user','categories')->get();

في هذه الحالة يتم تحميل البيانات المرتبطة مسبقًا بعدد أقل من الاستعلامات

- تحسين الأداء.
- تقليل عدد الاستعلامات.
- تسريع تحميل الصفحات.

• **Faker Library**

• هي مكتبة تُستخدم لإنشاء بيانات وهمية (Fake Data) لاختبار التطبيق أو ملء قاعدة البيانات أثناء التطوير.

• **توليد بيانات وهمية لاختبار التطبيق وملء قاعدة البيانات أثناء التطوير ⇒ Faker**

• بدلاً من إدخال البيانات يدويًا، تقوم Faker بإنشاء بيانات تشبه البيانات الحقيقية.

بنستخدم Faker؟

- اختبار التطبيق بكمية كبيرة من البيانات.
- تجربة شكل الموقع قبل إدخال البيانات الحقيقية.
- توفير الوقت أثناء التطوير.
- اختبار عمليات البحث والترتيب وال Pagination.

استخدامات Faker

- إنشاء أسماء عشوائية. `fake()->name()`
- إنشاء بريد إلكتروني عشوائي. `fake()->email()`
- إنشاء أرقام هواتف. `fake()->phoneNumber()`
- إنشاء عناوين. `fake()->paragraph()`
- تاريخ عشوائي. `fake()->date()`
- إنشاء نصوص وجمل تجريبية. `fake()->sentence()`
- ```
public function definition(): array
{
 return [
 'title' => fake()->sentence(),
 'content' => fake()->paragraph(),
];
}
```
- ال Factory تُستخدم مع Faker لإنشاء بيانات وهمية تلقائياً للنماذج (Models).

## Factory

- ال Factory هي المكان الذي نحدد فيه شكل البيانات الوهمية التي سيتم إنشاؤها بواسطة `Faker`.
- 
- 
- مثال: يقوم بإنشاء 10 سجلات Posts ببيانات عشوائية.
- ```
Post::factory(10)->create();
```
-
- **Seeder :**
- ال Seeder مسؤول عن إدخال البيانات إلى قاعدة البيانات
- `php artisan db:seed` to run seeder